# Does Restoration Really Help Detection?

A Deep Dive

**Name:** Shourja Banerjee

**Roll No.:** 24155423 **Section:** IPA 9

**Subject:** Image Processing and Applications

Code: github.com/onigiri4711/does-restoration-help-detection

November 2026

**Abstract**

Image restoration is usually judged by PSNR, but restored images are increasingly used by detectors rather than people. I asked whether restoration helps a pretrained object detector, and whether PSNR picks the restorer that helps most. In stage 1, 300 COCO val2017 images were damaged with Gaussian noise, JPEG compression and low light, restored with tuned classical filters, BM3D, CLAHE, gamma correction and a DnCNN trained from scratch, and passed to YOLOv8n. A strong denoiser recovered about three-quarters of the mAP lost to noise (DnCNN 77% at $\sigma = 25$), BM3D tied DnCNN for detection despite lower PSNR, DnCNN did not help on JPEG, and in low light PSNR ranked the methods in the wrong order. In stage 2, damage measured inside YOLOv8n showed that DnCNN removes 82–93% of the pixel error but only 58–62% of the feature damage at the neck. In stage 3, I fine-tuned DnCNN with an added detector-feature loss. At $\sigma = 50$ this raised YOLOv8n mAP by 0.010 over a pixel-only control with the $\lambda$ chosen on validation images (0.015 with the best $\lambda$ on the test images), and YOLOv8s, which was never used in training, also improved. In these experiments a higher PSNR did not always mean better detection: which restorer helped most depended on the type of damage and on how the detector responded to it.

# Table of Contents

**Abstract** **1**

**1 Introduction** **3**

**2 Literature Review** **3**

**3 Novelty and Contribution** **4**

3.1 What is new....4

3.2 Contributions....5

**4 Proposed Methodology** **5**

4.1 Stage 1: Degrade → Restore → Detect....6

4.2 Stage 2: looking inside the detector....9

4.3 Stage 3: denoiser trained on detector features....10

4.4 Hypotheses and stages....12

**5 Data** **12**

**6 Evaluation Metrics** **13**

6.1 Restoration quality metrics....13

6.2 Detection accuracy....14

6.3 Feature-space measures....14

6.4 Statistics....15

**7 Comparison with Existing SOTA** **15**

7.1 Against published numbers....15

7.2 Stage 1: does restoration help, and does PSNR pick the right restorer?....16

7.3 Stage 2: inside the detector....20

7.4 Stage 3: a denoiser trained on detector features....21

**8 Conclusion and Remarks** **24**

8.1 Conclusions....24

8.2 Limitations and Future Work....26

**References** **27**

**Use of AI Assistance** **27**

# 1 Introduction

Why should a restorer model that scores best only on pixels be the one that helps the detector most? I find denoising the most intriguing and curious topic in image processing, and it was my main motivation for this project.

Conventionally, the images reaching vision systems are damaged through sensor noise, JPEG compression or low light. The main aim of restoration through classical filters like BM3D or learned networks like DnCNNs is to undo the damage done in an image. This restoration is then judged by PSNR, i.e. peak signal to noise ratio, it measures how close the resulting pixels are to the original image. It remains unclear, however, if a high PSNR translates to something better for the detector. A detector rather relies on detecting edges and textures of the image, not the raw pixel values. The restorer model might win on the PSNR metric but might still remove the details the detector needs. For example, smoothing raises PSNR but at the cost of losing the finer details the detector may need.

Does restoration really help the detector at its job? Is it PSNR that picks the restorer that will help the detector out the most? Now, with all that in mind, if PSNR is indeed misleading, then maybe the damage measured in the feature maps of the detector helps track the detection loss better. In the final part of my research, I check if training a denoiser to repair those features helps detection, including on detectors it never saw?

In the initial stage of my project, I first built the pipeline for degrade-restore-detect. I used three kinds of damage, then some classical filters, block matching and 3D filtering (BM3D), CLAHE, DnCNN trained from scratch, as well as a brighten and denoise pipeline. Finally, a pretrained YOLOv8n was scored on the COCO mAP evaluation metric on a random fixed subset of 300 images in COCO val2017. Here all restorers get the identical damaged image, so their comparisons are paired. In the next stage, I hook every layer of YOLOv8n and measure, layer by layer, how far a damaged or restored image's features are from the clean images. Here no training was done. Then I fine-tune the DnCNN with a loss that adds a feature term to the usual pixel loss. Then I compare it against a control without the feature term. The detectors YOLOv8s and Faster R-CNN, which were never used in training, are also tested. The hypotheses were written down before each experiment was run, and the differences were tested with paired bootstrap 95% confidence intervals.

The rest of this report reviews related work (Section 2), states what is new (Section 3), describes the methods, data and metrics (Sections 4–6), compares the results with published work (Section 7) and concludes (Section 8).

# 2 Literature Review

Classical denoisers remove noise by exploiting the structure of natural images. Linear smoothing and the median filter average or rank neighbouring pixels, while edge-preserving filters such as the bilateral filter average only neighbours of similar

intensity. BM3D (Dabov et al., 2007) goes further: it groups similar 2-D image patches into 3-D arrays and filters each group jointly in a transform domain, a step its authors call collaborative filtering, so that fine detail shared by the grouped patches is kept while the noise is removed. It has long served as the reference classical denoiser and remains the standard non-learned baseline.

DnCNN (Zhang et al., 2017) showed that a deep convolutional network can outperform such methods. Instead of predicting the clean image, it predicts the noise and subtracts it (residual learning), and it uses batch normalisation to train a deep plain network. Trained over a range of noise levels, a single model handles blind Gaussian denoising, where the noise level is unknown, and on the BSD68 benchmark it outperforms BM3D by about 0.6 dB at $\sigma = 50$. Like most of this literature, DnCNN is evaluated with pixel-fidelity metrics such as PSNR and SSIM (Wang et al., 2004), which measure how close the restored image is to the clean one rather than how useful it is to a later model.

Liu et al. (2018) connected restoration and recognition directly. They placed a denoiser in front of a fixed classification or segmentation network and trained the denoiser with a joint loss: the usual pixel reconstruction error plus the downstream task's own loss. A denoiser trained this way gave higher classification accuracy and segmentation mIoU than one trained for pixels alone, especially at high noise, and still helped when used with the other task. Their results show that the denoiser preferred by pixel metrics is not necessarily the best one for a downstream model. Their experiments cover classification and segmentation under Gaussian noise, and they do not check whether PSNR ranks restorers in the same order as the task metric. I wanted to test this for object detection, where the detector relies on features that may react differently to noise, JPEG compression and low light, and to look inside the detector before training a denoiser for it.

# 3 Novelty and Contribution

## 3.1 What is new

The closest earlier work is Liu et al. (2018), who trained a denoiser with a downstream task loss for classification and segmentation under Gaussian noise. The main difference in this project is the task: I studied object detection, under three kinds of damage (Gaussian noise, JPEG compression and low light) instead of noise alone. I also wanted to know whether PSNR actually predicts detector performance, so I compared the two rankings directly, across conditions and per image. That question led to stage 2, where I measured layer by layer where the damage enters the detector before deciding how to train the final denoiser; the layer for the feature loss was then chosen from this analysis by a rule fixed in advance. To my knowledge, this combination has not been tested before. Throughout, the comparisons are paired, reported with bootstrap confidence intervals and checked against hypotheses written

down beforehand, and the fine-tuned denoiser is also tested on two detectors that were never used in training.

### 3.2 Contributions

1. A degrade–restore–detect pipeline with seeded damage models, nine automated checks and 64 restored runs on 300 COCO images (Sections 4.1 and 7.2).
2. Results showing where pixel quality and detection disagree: the BM3D–DnCNN tie, DnCNN on JPEG and the PSNR reversal in low light (Section 7.2).
3. A layer-by-layer feature analysis of YOLOv8n, comparing how much damage each restorer removes from the pixels and from the features (Section 7.3).
4. A DnCNN fine-tuned with a detector-feature loss, compared against a pixel-only control trained in the same way and tested on three detectors (Section 7.4).
5. Components built from scratch: the DnCNN itself, and a JPEG codec and a convolution layer written in NumPy and checked against reference implementations (Sections 4.1.6 and 7.1).

Hypotheses that failed, including near-misses, are reported as failures (Sections 7.2–7.4).

## 4 Proposed Methodology

**Table 1:** The three stages of the project.

| Stage | Project | The question | What was done |
|---|---|---|---|
| S1 | Degrade → Restore → Detect Notebooks(01-06) | Does restoration help a detector, and does PSNR pick the restorer that helps most? | Damaged images, restored them with filters, BM3D, CLAHE, DnCNN and pipelines, then scored PSNR and YOLOv8n mAP |
| S2 | Restoring for the Machine, Part A (notebook 07) | Where does damage show up inside the detector, and does feature distance track detection better than PSNR? | Hooked every YOLOv8n layer and measured the distance from clean features. No training |
| S3 | Restoring for the Machine, Part B (notebook 08) | Does a denoiser trained to repair detector features improve detection, including on detectors it never saw? | Fine-tuned DnCNN with pixel loss plus λ times a feature loss at layer 21; tested on YOLOv8n, YOLOv8s and Faster R-CNN |

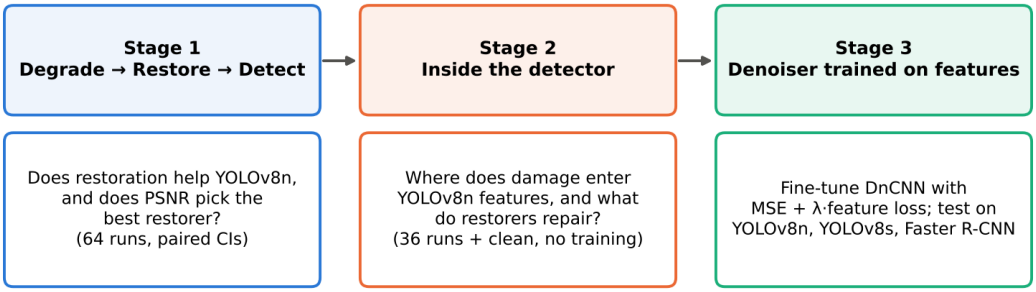



**Figure 1:** Overview of the three stages and how each leads to the next.

## 4.1 Stage 1: Degrade → Restore → Detect

### 4.1.1 Pipeline and setup

I divided the project into three stages (Fig. 1): the first tests whether restoration helps detection, the second examines what happens inside the detector, and the third uses those findings to train the denoiser. The images are float32 RGB with values within [0, 1]. Each damage is a function of the form clean image x, severity s, and a random generator. The result is then clipped to [0, 1] once, at the end after all damage steps are done. The random generator is seeded using the image index, the damage index and the severity index. This is used to replicate the same noise each run so that each restorer model gets an identical noisy image for paired comparison of the output. Here images are tested with both PSNR and SSIM on the 96 BSD500 images, and with mAP on the 300 COCO images. The stage 1 pipeline is shown in Fig. 2.

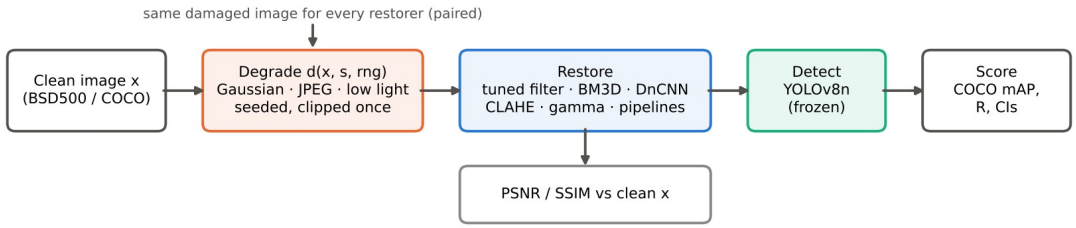



**Figure 2:** The stage 1 pipeline. Every restorer receives the identical seeded damaged image.

### 4.1.2 Damage models

**Table 2:** Damage models and severities.

| Damage | Model | Severities (mild → severe) |
|---|---|---|
| Gaussian noise | $y = x + n$, $n \sim N(0, (\sigma/255)^2)$, independent per pixel | $\sigma$ = 5, 15, 25, 35, 50 |
| JPEG | encode and decode at quality q | q = 90, 70, 50, 30, 10 |
| Low light | For description refer below | $\ell$ = 0.2 to 0.9 |

Poisson, salt and pepper, blur and colour cast are used only within the restoration study; for further details refer to notebook 01.

For the low light model I used a separate design, using the gamma curve transform I darken the image

x_d = $(1 - 0.8\ell) \cdot x\hat{}(1 + 2\ell)$. Then added a signal-dependent shot noise, with peak $200(1 - \ell) + 5$. After that add read noise, with standard deviation $0.01 + 0.04\ell$. If however $\ell = 0$ it means no change.

For detection however, only Gaussian noise ($\sigma$ = 15, 25, 35, 50), JPEG (q = 70, 50, 30, 10) and low light ($\ell$ = 0.4, 0.6, 0.8, 0.9) were considered (Table 2). Fig. 3 shows the Gaussian noise levels on one image.




**Figure 3:** Gaussian noise at $\sigma$ = 5, 15, 25, 35 and 50 on a test image (notebook 01).

### 4.1.3 Validation before experiments

A total of nine automated checks of the damage functions were run before any experiment.

The checks covered three areas: statistical properties (checks 2–4), expected behaviour (checks 5, 6, 8 and 9) and output format (check 1). All nine passed; the details are in notebook 01.

### 4.1.4 Restorer models

#### Tuned Classical Filters

Some tuned classical filters include Gaussian, bilateral, median and non-local means filters. Here each grid search is tuned on 4 validation images from the BSD500 (Berkeley Segmentation) dataset for every damage and severity then this is in turn applied to the rest 96 images. Hence these tuned filters are an oracle: they perform the best they can for each damage and level. A blind NL-means sets its strength based on a wavelet based noise estimate.

#### Block matching and 3D filtering (BM3D)

The original code has been used specifically for colour images using the bm3d_rgb. When the true sigma is provided it becomes an oracle. However for detection it was only used on Gaussian noise, it took about 1.8 hours per noise level on 4 CPU cores.

Contrast Enhancement - use of Global histogram equalization and CLAHE (Zuiderveld, 1994). Here Lab was used instead of RGB, for histogram it would require to equalise for each channel separately. Lab holds Lightness (L), green to red axis (a), blue to yellow axis (b). Here brightness only lives in L and colour lives only in a and b. With both HE and CLAHE, they were applied directly only to L, i.e. to the change in brightness and contrast.

Denoiser Convolutional Neural Network (DnCNN) [Zhang et al., 2017] (trained from scratch)

The type of DnCNN used was blind colour DnCNN. What the model does is predict the noise residual $\hat{v} = f(y)$ and output $y - \hat{v}$ (residual learning). The architecture is given in Table 3.

**Table 3:** The DnCNN architecture.

| Layer | Layer name with input to output channels | Parameters |
|---|---|---|
| 1 | Conv 3 → 64, ReLU | 1,792 |
| 2–16 | Conv 64 → 64 (no bias), batch norm, ReLU | 15 × 36,992 = 554,880 |
| 17 | Conv 64 → 3, no activation | 1,731 |

Total parameters: 558,403

Theoretical receptive field: 35 × 35 [as 1 + 17 × 2 = 35]

The training data was the 400 BSD500 train and test images, cut into roughly 381,000 patches of 40 × 40 with random flips and rotations per patch. Gaussian noise with $\sigma \sim U[0, 55]$ was added and the result clipped to [0, 1], and the error was calculated with MSE.

The optimizer used was Adam with a learning rate of $10^{-3}$, divided by 10 at 60% and 85% of 50 epochs, with batch size 128, on a T4 GPU on Kaggle for 3.8 hours.

The 100 BSD500 validation images, which contain the CBSD68 benchmark, were held out to measure denoising on unseen images. Four of them were used to tune the classical filters, so restoration PSNR and SSIM are reported on the other 96. JPEG and low-light images were never used in training.

Low-light pipelines

Combined in both orders: gamma → DnCNN and DnCNN → gamma, plus DnCNN → CLAHE and CLAHE → DnCNN.

A blind gamma curve raises every channel to the power $g = \log 0.45 / \log \bar{Y}$, where $\bar{Y}$ is the image's mean luma.

The main purpose here is to test whether contrast enhancement fails because it amplifies the noise.

### 4.1.5 Detection Model protocol

Detector: pretrained YOLOv8n (Jocher et al., 2023) with no fine tuning

Data: fixed random subset of 300 COCO val2017 images with 2,035 objects (808 small, 697 medium, 530 large).

Evaluation Metric: official COCO mAP@[.5:.95].

**Setting for detection:**

The confidence threshold is intentionally kept down to $10^{-3}$, so that mAP can integrate the full precision recall curve. YOLO's 80 class indices are directly mapped to COCO's non-contiguous category ids.

Each image is damaged with the same functions and seeds as the restoration experiments. The restorers run for each damage type are listed in Table 4.

**Restorers run per type of damage:**

**Table 4:** Restorers run for each damage type in the detection experiments.

| Damage | Restorers |
|---|---|
| Gaussian | none, tuned filter, BM3D, DnCNN |
| JPEG | none, tuned filter (bilateral), DnCNN |
| Low light | none, tuned filter, DnCNN, CLAHE, gamma, and the four pipelines |

#### 4.1.6 Built from scratch

I implemented a JPEG codec and a convolution layer from scratch by coding it in NumPy then verified it against libjpeg (within 0.05 dB on the test image, up to quality 90) and numerical gradients respectively.

Repo: https://github.com/onigiri4711/does-restoration-help-detection

JPEG codec: stage1_degrade_restore_detect/notebooks/02_jpeg_from_scratch.ipynb

Convolution layer: stage1_degrade_restore_detect/notebooks/03_numpy_conv_gradcheck.ipynb

### 4.2 Stage 2: looking inside the detector

In the earlier stage we saw that PSNR was misleading, so this stage directly measures the damage happening inside the detector, with no training.

**Setup**

The model used is YOLOv8n pretrained on COCO weights, kept frozen, with the same detection settings as stage 1 (i.e. confidence 0.001, IoU 0.7, max 300 detections). It uses forward hooks on all layers except Upsample, Concat and Detect, as they don't help in computing new features but only resize and rejoin maps. The layers used were layers 0 to 9 (backbone and SPPF) and layers 12, 15, 16, 18, 19 and 21 (neck), along with the input itself. These forward hooks copy each layer's output as the image goes from layer to layer. Layers 15, 18 and 21 are the neck outputs: the P3, P4 and P5 feature maps that feed the Detect head.

After this setup, I compared each feature map to the clean image's map. The letterbox padding that YOLO adds was cropped away first. This was done by using cosine distance, which compares the direction of the damaged and clean channel vectors at each position. Here zero would mean the same pattern, and larger means a

different pattern. I have also used centered kernel alignment (Kornblith et al., 2019) and relative squared L2 error as further checks. The neck distance was the average cosine distance over layers 15, 18 and 21. It was fixed before running, so that there was no picking the layers that looked best afterwards. The hooked layers are shown in Fig. 4.

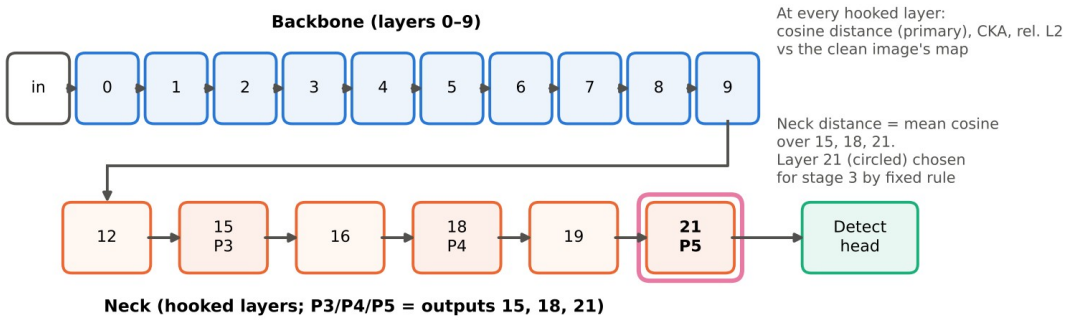



**Figure 4:** The hooked layers of YOLOv8n. The backbone (layers 0–9) feeds the neck, whose outputs 15, 18 and 21 (P3, P4 and P5) go to the detection head. Layer 21 (circled) was chosen for stage 3.

All in all there were 36 damaged runs plus the clean reference; each run is a damage type at a specific level with a given restorer, applied across all 300 test images with the same damage seeds as stage 1. The runs cover Gaussian noise ($\sigma$ = 15, 25, 50), JPEG (q = 30, 10) and low light ($\ell$ = 0.6, 0.8), with a subset of the stage 1 restorers; BM3D was run at $\sigma$ = 25 and 50 only. The dark-only runs ($\ell$ = 0.8, with no restorer, gamma, CLAHE and DnCNN) separate the effect of darkness from that of noise. The second noise seed at $\sigma$ = 25 measures how repeatable per-image AP loss is, as a benchmark for the per-image correlations.

This stage asked four questions: where does the damage show up as it moves through the layers? Does DnCNN really repair the features as well as the pixels? Why does CLAHE fail in low light? And does feature distance predict detection better than PSNR?

Through stage 2, I selected a layer for stage 3 using a rule fixed in advance. The candidates were the neck mean, the mean of layers 0–4 and the best single layer, each measured with cosine distance or relative L2. The best single layer was chosen on half the images, and every candidate was then scored on the other half by its correlation with AP loss on the Gaussian runs. Cosine distance at layer 21 scored highest and was chosen. If every candidate had scored below $\rho$ = 0.15, stage 3 would have fallen back on YOLO's own detection loss; this was not needed.

### 4.3 Stage 3: denoiser trained on detector features

Stage 2 showed that DnCNN repairs pixels much better than features, so in this final stage I fine-tuned it with the selected feature distance as an extra loss. This also tests whether repairing features improves detection, including on detectors it was never trained against.

### Model

Starting from the stage 1 DnCNN, with YOLOv8n frozen and in eval mode, the gradients flow through YOLOv8n into the denoiser only.

### The loss

$$L = MSE / MSE_0 + \lambda \cdot D / D_0$$

MSE is the pixel error between the denoised and clean crops. D is the cosine distance at layer 21 between YOLO's features for the denoised and clean crops, chosen by stage 2's rule. The MSE and cosine distance are on different scales: the feature term is about 40 times larger to begin with, so without normalisation even a small $\lambda$ would let the feature term dominate, and $\lambda = 1$ wouldn't imply equal weight. Each term is therefore divided by its starting value, and $\lambda$ is the weight of the feature term relative to the pixel term. $MSE_0$ and $D_0$ are the starting DnCNN's average MSE and D on 20 fixed training batches (0.0018 and 0.0731), and the same values are used for every $\lambda$. The training loop is shown in Fig. 5.

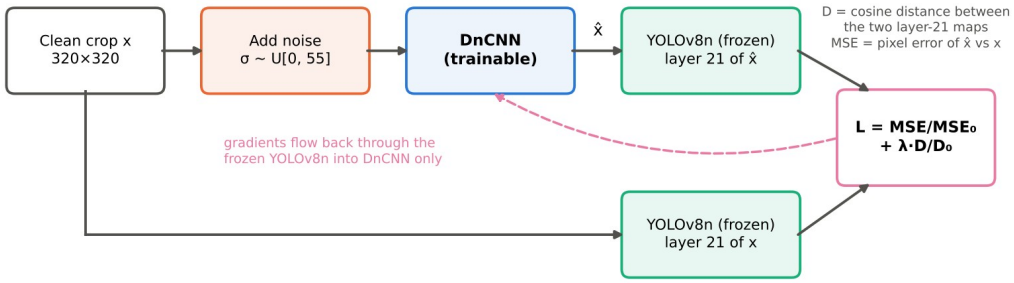



**Figure 5:** Stage 3 training. YOLOv8n is frozen; the loss combines the pixel error and the layer-21 feature distance, and only DnCNN is updated.

### Data and training

4,700 COCO val2017 images not in the 300-image test set were used: 4,200 for training and 500 for choosing $\lambda$. Noise was blind Gaussian noise with $\sigma \sim U(0, 55)$ per crop, as in stage 1. The crops were random 320 × 320 crops with horizontal flips, in batches of 8. Adam was used as the optimiser with learning rate $10^{-4}$ for 2,000 steps, with the rate × 0.1 for the last 500. For stability, gradient clipping was used at 1.0. The same seed was used for every model, hence each model sees the same crops and noise in the same order. It trained for about 39 minutes per model on a T4 GPU on Kaggle.

The $\lambda$ values were 0, 0.03, 0.1, 0.3, 1 and 3. Here $\lambda = 0$ is the control, as it separates the effect of the feature term from the effect of more training on COCO. The control uses the same setup, with the same data, steps and seed, but pixel loss only. A seventh model used the neck mean (layers 15, 18 and 21) instead of layer 21, at $\lambda = 1$. $\lambda^*$ was chosen by a rule fixed in advance: the $\lambda > 0$ with the highest YOLOv8n mAP on the 500 validation images at $\sigma = 50$, among those whose

validation PSNR is at most 1 dB below the control's. The test images were not used for this.

**Testing**

Testing used the same 300 images and noise seeds as stages 1 and 2, at $\sigma = 0$ (clean), 15, 25 and 50. PSNR was measured on the images, and mAP on three detectors: YOLOv8n, the one in the loss; YOLOv8s, the same family but bigger (near transfer); and Faster R-CNN (Ren et al., 2015) ResNet-50-FPN v2 (torchvision, COCO weights), a completely different design never used in training (far transfer), so a gain on it cannot come from just fooling YOLOv8n. Each model was compared against the control with a paired bootstrap of image-averaged AP with 2,000 resamples. The share of the restoration error's energy above half the Nyquist frequency was also measured, since patterns made to trick a network tend to be high-frequency.

### 4.4 Hypotheses and stages

The hypotheses were written down, with fixed thresholds, before each stage's experiments. They are summarised below; the full wording and thresholds are in the project repository: stage1_degrade_restore_detect/paper/paper.pdf for stage 1, and stage2_3_restoring_for_the_machine/PLAN.md (Sections 1.3 and 2.2) for stages 2 and 3. Section 7 reports the outcome of each.

**Stage 1 (S1-H1 to S1-H5).** mAP falls steadily as damage gets worse (H1); the restorers recover much of the loss on Gaussian noise, DnCNN most (H2), but little on JPEG (H3); CLAHE helps most in low light (H4); and in some condition the best-PSNR restorer is not the best-mAP one (H5).

**Stage 2 (S2-H1a to S2-H6, 13 hypotheses).** Where the damage appears across the layers (H1a, H1b); how much neck damage DnCNN removes compared with pixel error and mAP recovery (H2a–H2c); whether noise or darkness drives the low-light loss, and how CLAHE, gamma and gamma → DnCNN behave (H3a–H3d); whether neck distance predicts per-image AP loss and gain better than PSNR (H4a, H4b); whether BM3D matches DnCNN at the neck (H5); and whether neck distance picks the best restorer (H6).

**Stage 3 (S3-H7 to S3-H10).** At $\sigma = 50$, the chosen $\lambda^*$ raises YOLOv8n mAP by at least 0.010 over the control at a cost of no more than 1 dB PSNR (H7), and Faster R-CNN keeps at least half that gain (H8); mAP peaks at a $\lambda$ below the largest (H9); and $\lambda^*$ does not hurt clean images (H10).

The differences are tested with paired bootstrap confidence intervals described later in Section 6.4.

## 5 Data

The project uses two image datasets and three pretrained detectors, shown in Table 5 below.

**Table 5:** Datasets and detectors used.

| Data | What it is | From where | How it is being used |
|---|---|---|---|
| **BSDS500** (Arbeláez et al., 2011) | 500 natural photos (200 train, 100 val, 200 test) | Berkeley's dataset page (attached in the Kaggle notebook as an input) loaded from balraj98/berkeley-segmentation-dataset-500-bsds500 | 400 train + test images to train DnCNN (stage 1). The 100 val images held out: 4 for tuning the filters, 96 for testing restoration (PSNR, SSIM) |
| **COCO val2017** (Lin et al., 2014) | 5,000 photos with box labels for 80 object classes | cocodataset.org | A fixed random 300-image subset (2,035 objects) for every detection test in all three stages. 4,700 of the remaining images for stage 3: 4,200 for training, 500 for choosing $\lambda$ |
| **YOLOv8n weights** | Detector pretrained on COCO | Ultralytics | The main detector in all stages; frozen inside stage 3's loss |
| **YOLOv8s weights** | Larger detector from the same family | Ultralytics | Stage 3 test only (near transfer) |
| **Faster R-CNN ResNet-50-FPN v2** | A different detector architecture, pretrained on COCO | torchvision | Stage 3 test only (far transfer; never used in training) |

All the damage is generated from clean images with fixed seeds; no real damaged photographs were used. The DnCNN's training images don't overlap the 96 BSD500 test images; these were checked by filename and pixel hash. In stage 3 the 4,700 COCO images exclude the 300 test images.

For reproducibility of the data, the image ids of the 300-image test set and stage 3's training and validation split are saved in the repositories and all damage is seeded, so at each run the same data can be rebuilt the same way.

# 6 Evaluation Metrics

## 6.1 Restoration quality metrics

### Peak Signal-to-Noise Ratio (PSNR)

Here higher is better, measured in dB. A gain of 6 dB halves the typical pixel error. Identical images will give infinity.

$$\text{PSNR} = 10 \cdot \log_{10}(\text{MAX}^2 \,/\, \text{MSE})$$

where,

MSE is the mean squared error between the restored and clean pixels.

MAX = 1, because the pixel values are in [0, 1].

**Structural Similarity Index Measure (SSIM)**

It compares small windows of the two images on three things: brightness (luminance), contrast, and structure (the pattern of light as well as dark). 1 implies identical images, and lower values imply less similar images.

Both PSNR and SSIM are calculated on the held out 96 BSD500 images, with PSNR in float64. PSNR is also measured on the 300 COCO images in the detection runs. PSNR and SSIM aren't designed to predict a detector's accuracy.

### 6.2 Detection accuracy

**IoU (intersection over union) tells us how much a predicted box overlaps a true box, from 0 (i.e. no overlap) to 1 (meaning perfect). A prediction will be counted as correct if its IoU with a true box is above a certain threshold. Precision is the fraction of predictions which are accurate, and recall is the share of the total true objects that were found. AP (average precision) is the area under the precision–recall curve as the confidence threshold is lowered. It tells us how good the detector is at every confidence level. AP is also reported for small, medium and large objects: small if less than $32^2$ pixels, large if greater than $96^2$ pixels, and medium in between.**

mAP@[.5:.95] is the AP averaged over 10 IoU thresholds (0.50, 0.55 … 0.95) and over all 80 classes (computed with the official COCOeval code). Averaging over strict thresholds helps to reward boxes that fit tightly, not just roughly.
The recovered share (R) can be expressed as follows:

$$R = (\text{mAP_restored} - \text{mAP_damaged}) \,/\, (\text{mAP_clean} - \text{mAP_damaged}).$$

Here, for the recovered share R, 100% means back to clean mAP, 0% implies it was of no help, and below 0 means the restorer made things worse for the detector.

### 6.3 Feature-space measures

For more information about cosine distance, CKA and relative L2, refer to Section 4.2.

Neck distance: this is the mean cosine distance over layers 15, 18 and 21 (refer to Section 4.2).

Feature-damage removal = 1 − d(restored) / d(damaged) at a layer, where d is the distance from the clean map; pixel removal is the same ratio using pixel MSE. Here 100% means the damage is fully repaired, 0% means no repair, and below 0 means the restored image is further from clean than the damaged one.

High-frequency share is the fraction of the restoration error's energy above half the Nyquist frequency, i.e. in the finest details; it is a check to look for adversarial-looking patterns.

### 6.4 Statistics

The COCO mAP pools detections across images, so it can't be cheaply recomputed on a resample. The bootstrap therefore uses image-averaged average precision i.e. the mean of each image's own average precision.

The image-averaged AP agrees with COCO mAP on how it ranks stage 1's 65 runs, with Spearman 0.997.

10,000 resamples (stage 1) or 2,000 resamples (stage 3) of the 300 images were drawn with replacement, using the same resamples for every run so the differences here are paired (in comparison). A difference is called clear when its 95% percentile interval excludes zero.

Spearman rank correlation ($\rho$) is the way of finding whether two quantities put items in the same order, i.e. 1 means the same order, 0 means unrelated and $-1$ means completely reversed. It is used for PSNR vs mAP across runs, for per-image PSNR gain vs AP gain, and for feature distance or PSNR vs per-image AP loss in stage 2.

The noise ceiling is the Spearman correlation between per-image AP losses from two different noise draws on the same set of images. It gives an upper reference for judging stage 2's correlations. No per-image measure can do better than this.

## 7 Comparison with Existing SOTA

This section first compares the restorers and the detector with published numbers, then reports the three stages in turn. A difference is called clear when its 95% paired bootstrap interval excludes zero (Section 6.4).

### 7.1 Against published numbers

**Table 6:** Restoration quality, PSNR in dB. Ours on the 96 held-out BSD500 images with clipped noise; published values on CBSD68 from Zhang, Zuo and Zhang (2018, FFDNet), Table V. The tuned filter is the best classical filter for each $\sigma$.

| $\sigma$ | Noisy | Tuned filter | Our BM3D | Published CBM3D | Our DnCNN | Published CDnCNN-B |
|---|---|---|---|---|---|---|
| 15 | 24.82 | 30.90 | 33.20 | 33.52 | 33.76 | 33.89 |
| 25 | 20.53 | 27.56 | 30.18 | 30.71 | 31.05 | 31.23 |
| 50 | 15.00 | 23.20 | 25.92 | 27.38 | 27.69 | 27.92 |

The DnCNN trained here reproduces the published blind colour model (Table 6): it is 0.13–0.23 dB below the published CBSD68 values, on a closely related set of images (the 96 held-out BSD500 validation images, which contain CBSD68) and with clipped noise. The published model is deeper (20 layers, against 17 here). It therefore matches the published model rather than improving on it. BM3D is close to its published values at $\sigma = 15$ and 25 but 1.46 dB lower at $\sigma = 50$. The most likely cause is the clipping of the noise to [0, 1], which BM3D does not model, but this was

not tested. As a result, DnCNN's PSNR lead over BM3D at $\sigma = 50$ is larger here (1.77 dB) than in the published results (0.54 dB).

**Table 7:** Detector baseline, mAP@[.5:.95].

| | **Published** | **Ours** |
|---|---|---|
| YOLOv8n | 0.373 (full COCO val2017, Ultralytics docs) | 0.387 (our 300-image subset) |

YOLOv8n reaches 0.387 mAP on the 300-image subset (Table 7), close to the published 0.373 on the full COCO val2017 set; the small difference comes from evaluating on a subset.

## 7.2 Stage 1: does restoration help, and does PSNR pick the right restorer?

**Table 8:** YOLOv8n mAP@[.5:.95] on 300 COCO images, with the recovered share R in brackets. Clean mAP = 0.387. The tuned filter for JPEG is the bilateral filter.

| **Condition** | **None** | **Tuned filter** | **BM3D** | **DnCNN** | **CLAHE** | **gamma → DnCNN** |
|---|---|---|---|---|---|---|
| Gaussian $\sigma = 25$ | 0.241 | 0.305 (44%) | 0.343 (70%) | 0.354 (77%) | – | – |
| Gaussian $\sigma = 50$ | 0.088 | 0.169 (27%) | 0.297 (70%) | 0.304 (72%) | – | – |
| JPEG q = 30 | 0.309 | 0.346 (48%) | – | 0.305 (−5%) | – | – |
| JPEG q = 10 | 0.125 | 0.231 (41%) | – | 0.121 (−2%) | – | – |
| Low light $\ell = 0.6$ | 0.207 | 0.226 (10%) | – | 0.283 (42%) | 0.174 (−18%) | 0.305 (55%) |
| Low light $\ell = 0.8$ | 0.121 | 0.134 (5%) | – | 0.213 (35%) | 0.095 (−10%) | 0.242 (46%) |

**Table 9:** Low light at $\ell = 0.8$: PSNR and mAP disagree (PSNR on the COCO images).

| | **None** | **DnCNN** | **CLAHE** | **gamma** | **gamma → DnCNN** | **DnCNN → gamma** |
|---|---|---|---|---|---|---|
| PSNR (dB) | 8.13 | 8.07 | 10.37 | 12.81 | 16.17 | 16.71 |
| mAP | 0.121 | 0.213 | 0.095 | 0.059 | 0.242 | 0.196 |

Noise. On Gaussian noise, a strong denoiser recovers about three-quarters of the lost mAP (Table 8). DnCNN recovers 77% at $\sigma = 25$ and 72% at $\sigma = 50$, and BM3D about 70%, against 44% and 27% for the best tuned filter. DnCNN's lead over the tuned filter is clear from $\sigma = 25$ upward (+0.029, +0.057 and +0.146 image-averaged AP at $\sigma = 25$, 35 and 50) but not at $\sigma = 15$ (+0.010, interval −0.002 to +0.024). BM3D also clearly beats the tuned filter from $\sigma = 25$ upward (+0.021, +0.047 and +0.134). DnCNN and BM3D, however, are statistically tied: DnCNN's lead is only

0.004–0.013 and is not clear at any $\sigma$ (at $\sigma = 50$ the interval is $-0.000$ to $+0.027$). At $\sigma = 35$, BM3D even matches DnCNN (0.327 against 0.326) despite 1.3 dB lower PSNR. This suggests that the detection gain comes mainly from removing the noise well, rather than from using a learned denoiser. DnCNN's practical advantages are that it is blind and fast: about 45 s for the 300 images on a T4 GPU, against about 1.8 hours per noise level for BM3D on four CPU cores.

JPEG. DnCNN never helps on JPEG ($-5\%$ at q = 30 and $-2\%$ at q = 10); it was never trained on block artifacts. The bilateral filter recovers 48% of the lost mAP at q = 30 and 41% at q = 10 by smoothing the $8 \times 8$ block edges, which the detector would otherwise treat as real edges. At q = 50 and 70 its gain is not clear. So in these runs, removing JPEG artifacts only paid off at strong compression.

Low light. In low light, PSNR ranks the methods in the wrong order (Table 9). CLAHE makes detection worse than doing nothing at every level ($-18\%$ at $\ell = 0.6$ and $-10\%$ at $\ell = 0.8$, clear at all four levels), although among the single methods it has the highest PSNR; gamma brightening alone does the same (0.059 against 0.121 unrestored at $\ell = 0.8$). DnCNN, although it never saw low-light images, is the best single method (42% and 35%). Brightening first and then denoising is best overall (55% and 46%) and clearly better than DnCNN alone ($+0.013$ to $+0.030$). The order of the two steps also changed the result. At $\ell = 0.8$, gamma before DnCNN gave 0.242 mAP against 0.196 for DnCNN before gamma, which I did not expect, since the second pipeline has the higher PSNR (16.71 against 16.17 dB). A likely reason is that brightening first lifts the noise into the range DnCNN was trained on, while denoising first leaves faint noise that the gamma curve then amplifies.

Does PSNR predict detection? Across all 64 restored runs, PSNR and mAP correlate (Spearman 0.76), but mostly because heavier damage lowers both. Within a single condition, which is what matters when choosing a method, PSNR and mAP order the restorers the same way on Gaussian noise (1 of 24 pairs reversed, the BM3D–DnCNN near-tie at $\sigma = 35$) and on JPEG (2 of 12), but reverse 63 of 144 pairs in low light. For individual images PSNR says almost nothing: across 52 restored runs, the Spearman correlation between a restorer's PSNR gain on an image and its AP gain on that image lies between $-0.14$ and $+0.21$ (median 0.03). Even at $\sigma = 50$, where DnCNN improves AP on 91% of the images, it lowers AP on 5%, and nothing in the pixels shows which ones. This is what led me to look inside the detector in stage 2.

Of the five stage 1 hypotheses, S1-H1 is confirmed (mAP falls at every step, to 0.088 at $\sigma = 50$, 0.125 at q = 10 and 0.056 at $\ell = 0.9$), S1-H2 is confirmed with a qualification (DnCNN beats the tuned filter but ties BM3D), S1-H3 is confirmed except at strong compression ($q \leq 30$), S1-H4 is rejected (CLAHE hurts detection in low light) and S1-H5 is confirmed (63 of 144 low-light pairs reversed).

I also analysed the trained DnCNN to understand these results. Of its 64 first-layer filters, 43 act mainly on differences between colour channels. Its effective receptive field widens with the noise, from $\pm 7$ pixels at $\sigma = 15$ and 25 to $\pm 10$ at $\sigma =$

50, and it keeps more fine detail than the best classical filter: its half-power frequency is 0.355, 0.254 and 0.152 cycles per pixel at $\sigma = 15$, 25 and 50, against 0.293, 0.137 and 0.098. On damage it never saw, such as JPEG and blur, it produced streaks along strong edges, which may explain why it does not help on JPEG.

Figure 6 shows mAP against severity for each damage type, Figure 7 all the low-light restorers and pipelines, Figure 8 PSNR against mAP for all 64 restored runs, and Figure 9 the paired bootstrap intervals for the key comparisons.

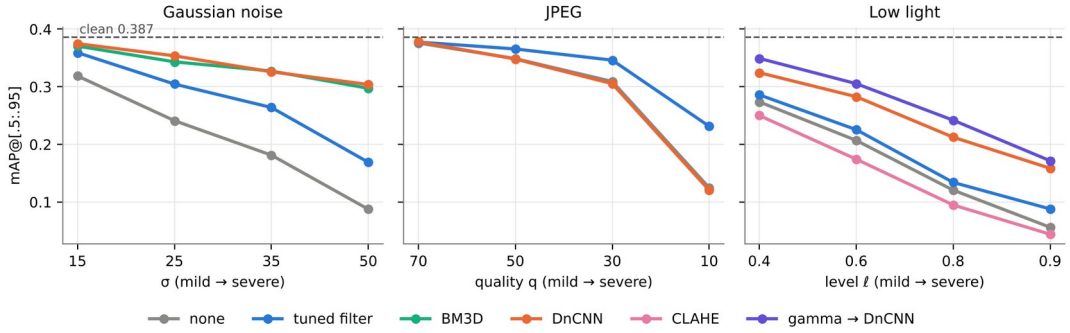



**Figure 6:** YOLOv8n mAP@[.5:.95] against severity for each damage type and restorer on the 300 COCO images (plotted from results/05b/sweep.csv). Dashed line: clean mAP.

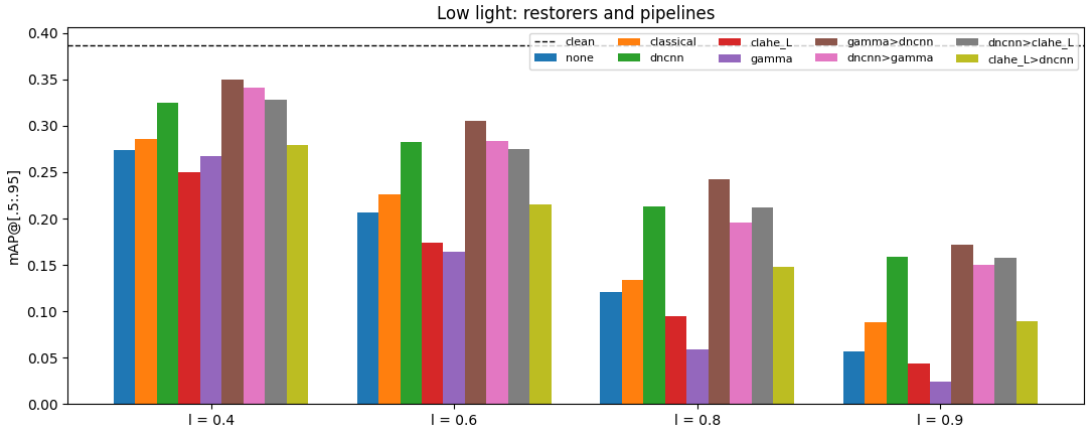



**Figure 7:** Low light: mAP of every single restorer and pipeline at each level $\ell$ (notebook output).

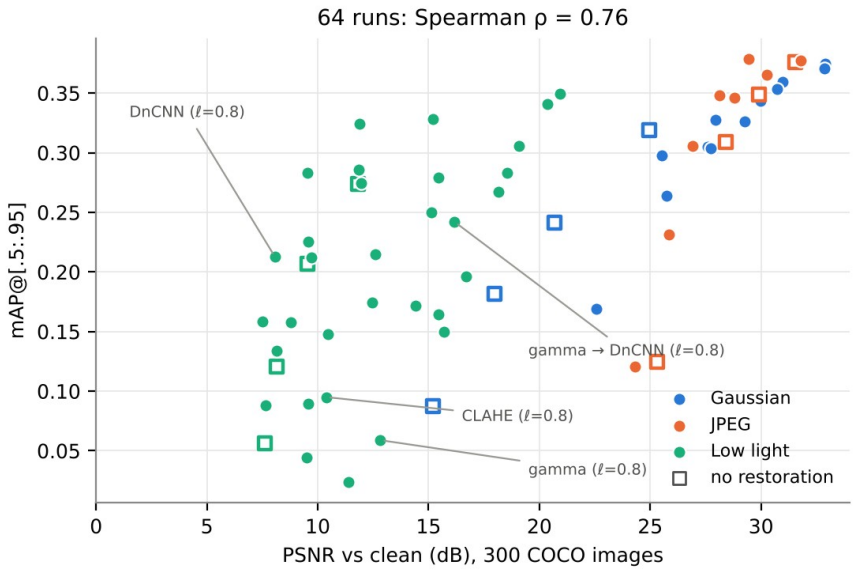



**Figure 8:** PSNR against mAP for all 64 restored runs, by damage type; hollow squares are unrestored images (plotted from results/05b/sweep.csv).

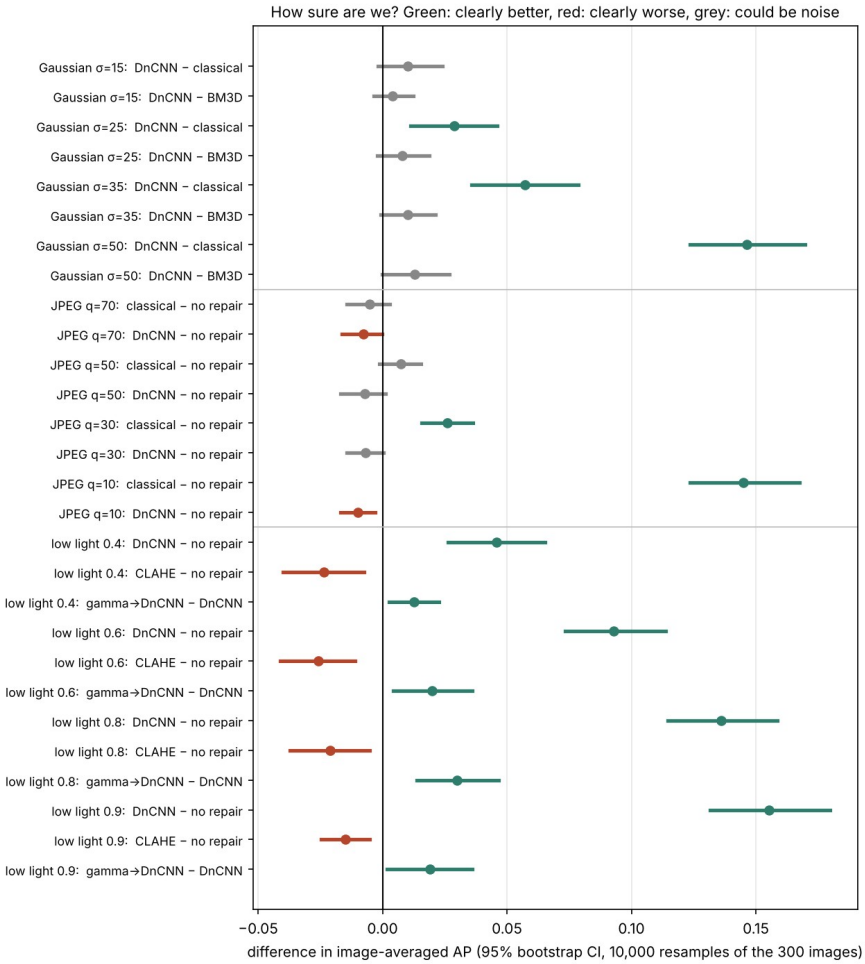



**Figure 9:** Paired bootstrap 95% intervals of the difference in image-averaged AP (10,000 resamples of the 300 images). Green: clearly better; red: clearly worse; grey: not clear.

### 7.3 Stage 2: inside the detector

**Table 10:** DnCNN on Gaussian noise: share of pixel error removed, share of neck damage removed, and mAP recovered (stage 2 runs).

| σ | Pixel error removed | Neck damage removed | mAP recovered |
|---|---|---|---|
| 15 | 82% | 58% | 82% |
| 25 | 88% | 62% | 77% |
| 50 | 93% | 59% | 72% |

Stage 2 reproduces the stage 1 mAP of every shared run to within 0.0002. The detector did not filter the damage out as it went deeper. For Gaussian noise the cosine distance peaks at layer 1 ($\sigma = 25$) and layer 7 ($\sigma = 50$), and the neck is about as damaged as the first layer (0.109 against 0.089 at $\sigma = 25$), so S2-H1a is not supported. JPEG damage grows with depth: at q = 10 the neck distance is 3.33 times the layer-0 distance, against 0.94 for Gaussian noise at $\sigma = 50$ (S2-H1b supported).

DnCNN repairs pixels far better than features (Table 10). It removes 82–93% of the pixel error but only 58–62% of the neck damage, and its neck repair is well below its mAP recovery of 72–82%, so YOLOv8n tolerates part of the damage that remains. This supports S2-H2b; S2-H2a narrowly fails (58% and 59% at $\sigma = 15$ and 50, against a threshold of 60%) and S2-H2c is not supported. DnCNN still leaves roughly 40% of the neck damage behind, and this is what I targeted in stage 3.

In low light, the noise is the problem rather than the darkness. Darkened images without noise keep 0.301 mAP, against 0.121 with noise, and gamma alone brings them back to 0.383 (S2-H3b and S2-H3c supported). On real low light, CLAHE removes 47% and 40% of the pixel error at $\ell = 0.6$ and 0.8 but moves the early-layer features 20% and 14% further from clean (S2-H3a supported): it brightens the noise along with the image. Gamma → DnCNN is closest to clean at the neck at both levels (S2-H3d supported), matching its best mAP.

The feature distances also gave a different picture from PSNR. BM3D's neck distance is within 12% ($\sigma = 25$) and 15% ($\sigma = 50$) of DnCNN's, despite 0.75 and 2.17 dB lower PSNR, consistent with their tied mAP (S2-H5 supported). Ranking the restorers by neck distance picks the best-mAP restorer in all 7 conditions (rank agreement 0.995), while PSNR picks it in 6 and fails at $\ell = 0.8$ (rank agreement 0.77; S2-H6 supported). Per image, the correlation between neck distance and AP loss averages 0.299, against −0.08 for PSNR, and is higher than PSNR's in 35 of 36 runs; for per-image gains it is 0.257 against 0.056, higher in 24 of 24 pairs. Both narrowly miss the pre-registered threshold of 0.30, so S2-H4a and S2-H4b are not supported. The noise ceiling is 0.74, so much of the per-image variation is repeatable but not captured by any single-layer distance.

In total, 8 of the 13 stage 2 hypotheses are supported. The thresholds were fixed in advance, so the near-misses count as failures. The pre-registered rule chose cosine distance at layer 21 for stage 3 (held-out $\rho = 0.238$, against 0.237 for relative L2 at the neck and 0.228 for the neck mean).

Figure 10 shows where the damage appears inside YOLOv8n, and Figure 11 how much of it the tuned filter and DnCNN remove at each layer.

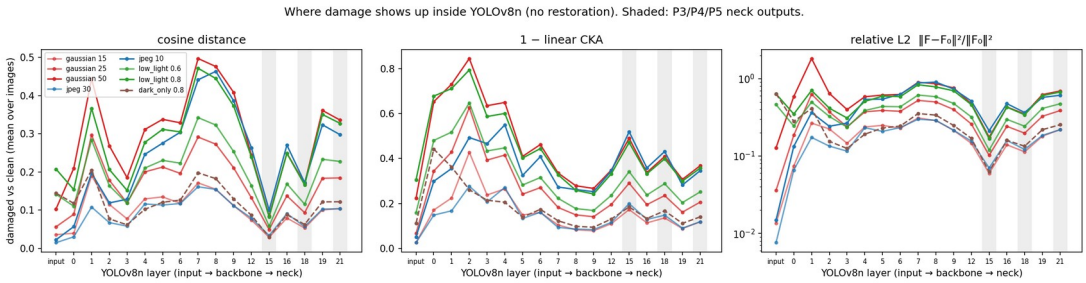



**Figure 10:** Damage inside YOLOv8n by layer with no restoration, measured as cosine distance, 1 − linear CKA and relative L2 to the clean features. Shaded: the neck outputs P3/P4/P5 (results/07).

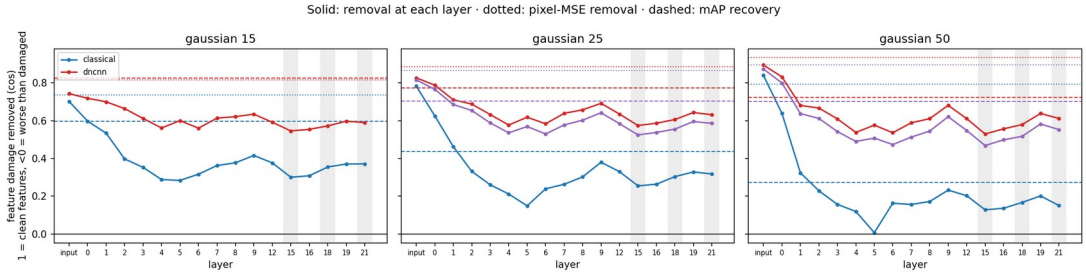



**Figure 11:** Share of feature damage removed at each layer by the tuned filter and DnCNN on Gaussian noise (solid), with pixel-error removal (dotted) and mAP recovery (dashed) for reference (results/07).

## 7.4 Stage 3: a denoiser trained on detector features

**Table 11:** Test mAP@[.5:.95] at $\sigma = 50$ (300 images). $\lambda^* = 1$ was chosen on the validation images.

| | **Start DnCNN** | **Control ($\lambda = 0$)** | **$\lambda = 0.03$** | **$\lambda = 0.1$** | **$\lambda = 0.3$** | **$\lambda = 1$ ($\lambda^*$)** | **$\lambda = 3$** | **Neck, $\lambda = 1$** |
|---|---|---|---|---|---|---|---|---|
| YOLOv8n (in the loss) | 0.304 | 0.302 | 0.312 | 0.314 | 0.317 | 0.312 | 0.309 | 0.317 |
| YOLOv8s (near transfer) | 0.365 | 0.361 | 0.366 | 0.369 | 0.370 | 0.368 | 0.365 | 0.371 |
| Faster R-CNN (far transfer) | 0.344 | 0.351 | 0.354 | 0.361 | 0.368 | 0.367 | 0.364 | 0.369 |
| PSNR (dB) | 27.72 | 27.62 | 27.62 | 27.58 | 27.53 | 27.39 | 27.19 | 27.44 |

At heavy noise, fine-tuning with the feature loss gave a small gain (Table 11). At $\sigma = 50$, every $\lambda$ above 0 beats the control for YOLOv8n, by 0.007 to 0.015 mAP, and the paired intervals are clear for every $\lambda$ except 0.03: $\lambda = 0.3$ gains +0.019 image-averaged AP (interval 0.008 to 0.029), $\lambda^* = 1$ gains +0.016 (0.004 to 0.028) and the neck variant +0.019 (0.007 to 0.030). The cost is 0.1–0.4 dB of PSNR, so PSNR and detection again move in opposite directions. The gain wins back about 16% of the mAP still missing after the starting DnCNN (0.304 → 0.317, against a clean 0.387).

The control scores the same as the starting DnCNN (0.302 against 0.304), so the gain comes from the feature term, not from the extra training on COCO.

I also tested the fine-tuned models on YOLOv8s and Faster R-CNN to see whether the gain carried over to other detectors. YOLOv8s, which was never used in training, gains clearly at every $\lambda$ (+0.011 to +0.019 image-averaged AP). Faster R-CNN, a different architecture, gains as much mAP as YOLOv8n or more (+0.016 at $\lambda^*$, +0.017 at $\lambda = 0.3$), but its per-image interval at $\lambda^*$ just includes zero (+0.008, −0.001 to 0.018); it is clear only at $\lambda = 0.1$. The high-frequency share of the restoration error is unchanged (0.550 for the control against 0.552 for $\lambda^*$), so there is no sign of adversarial patterns: the denoiser does not seem to be fooling YOLOv8n with patterns only it responds to.

The weakest part of stage 3 was choosing $\lambda$. On the 500 validation images all $\lambda$ values were within 0.01 mAP of each other, and $\lambda = 0.03$ and 0.1 even scored below the control, so the pre-registered rule picked $\lambda^* = 1$, while $\lambda = 0.3$ was best on the test images. This miss decides S3-H7: $\lambda^*$ raises YOLOv8n mAP by 0.0096, just below the 0.010 threshold, although its interval is above zero and its PSNR cost is only 0.24 dB. Choosing $\lambda = 0.3$ after seeing the test results would not be fair, so S3-H7 is not supported. S3-H8 is not supported because Faster R-CNN's interval includes zero. The mAP curve over $\lambda$ peaks in the middle, at $\lambda = 0.3$ (S3-H9 supported), and clean images are not harmed: every fine-tuned model keeps clean mAP within 0.003 of the control ($\lambda^*$: 0.390 against 0.388; S3-H10 supported). Fine-tuning on COCO also taught the denoiser to leave clean images almost untouched: clean-image PSNR rose from 37.5 dB to 39–46 dB. In total, 2 of the 4 stage 3 hypotheses are supported.

At $\sigma = 15$ and 25 the starting DnCNN is already within about 0.01–0.03 mAP of clean, and the differences there are small. A few are formally clear at $\sigma = 25$ (about +0.005 to +0.007 image-averaged AP for YOLOv8n and YOLOv8s at $\lambda = 0.03$ and 0.1, and for Faster R-CNN at $\lambda = 3$), but they are borderline and come from many comparisons, so they are not treated as a finding.

**Table 12:** Our method compared with Liu et al. (2018).

| | **Liu et al. 2018 (IJCAI)** | **Ours (stage 3)** |
|---|---|---|
| Task | Classification (VGG-16, ILSVRC2012 val), segmentation (Pascal VOC 2012 val) | Object detection (YOLOv8n, 300 COCO val2017 images) |
| Task term in the loss | The task's own loss (cross-entropy) | Feature distance at a layer chosen by the stage 2 analysis |
| Denoiser | Their own U-Net-style network | Our blind DnCNN, fine-tuned |
| Pixel-only comparison | Separately trained denoiser ("Separate + VGG") | Control: the same fine-tuning with $\lambda = 0$ |
| Result at the highest noise | $\sigma = 60$: Top-1 50.1 (separate) $\rightarrow$ 59.2 (joint); mIoU 46.59 $\rightarrow$ 52.02 (their Tables 2 and 3) | $\sigma = 50$: mAP 0.302 (control) $\rightarrow$ 0.312 ($\lambda^*$) and 0.317 ($\lambda = 0.3$) |
| Transfer test | Denoiser trained on one task, used on the other: Top-1 57.0, mIoU 51.82 at $\sigma = 60$ | Detectors never used in training: Faster R-CNN 0.351 $\rightarrow$ 0.367 at $\lambda^*$ |
| Uncertainty | None reported | Paired bootstrap intervals |

Compared with Liu et al. (2018), the closest earlier work, the approach is similar in spirit but differs in several ways (Table 12). They trained a denoiser with the task's own loss for classification and segmentation; this study uses a feature distance at a layer chosen by analysing the detector, and applies it to object detection. Both studies compare against a denoiser trained for pixels only: their joint training raises Top-1 accuracy at $\sigma = 60$ from 50.1 to 59.2 and segmentation mIoU from 46.59 to 52.02, while here mAP at $\sigma = 50$ rises from 0.302 to 0.312 ($\lambda^*$) or 0.317 ($\lambda = 0.3$). Their gain is larger, but the tasks, metrics and noise levels differ, so the numbers are not directly comparable. Liu et al. tested transfer to a different task; I tested transfer to a detector of a different architecture (Faster R-CNN) that was never used in training, and I report confidence intervals for every comparison.

Figure 12 shows PSNR against mAP for every $\lambda$ on the three detectors, and Figure 13 an example crop at $\sigma = 50$.

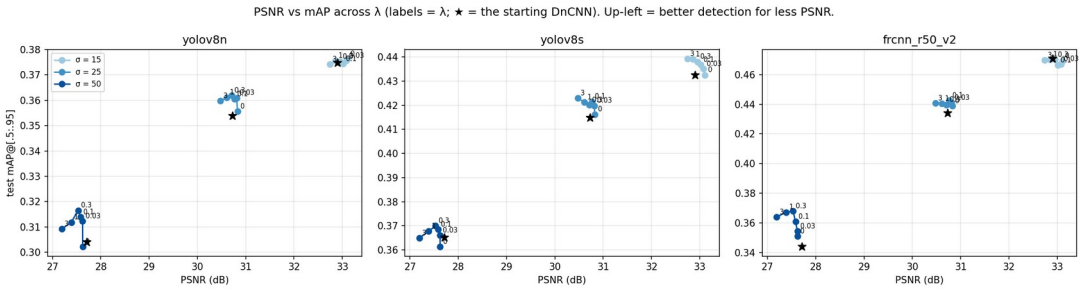



**Figure 12:** Test PSNR against mAP across $\lambda$ for YOLOv8n, YOLOv8s and Faster R-CNN at $\sigma = 15$, 25 and 50 (★ = starting DnCNN). Up and to the left means better detection for less PSNR (results/08).

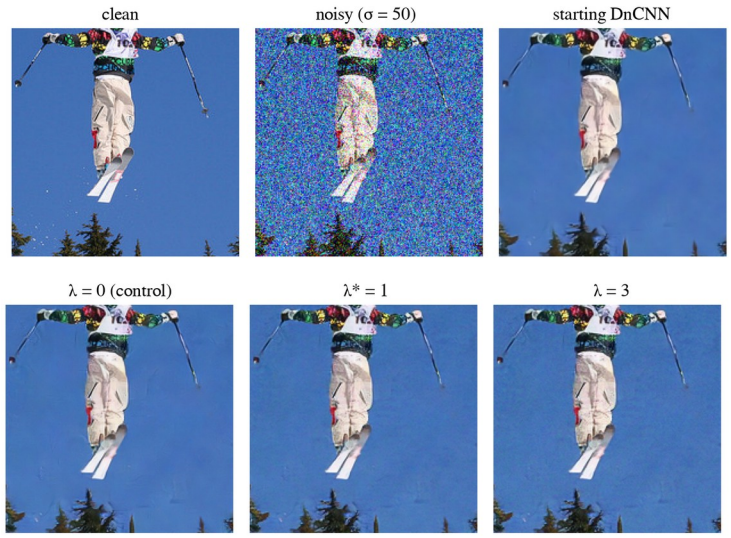



**Figure 13:** A test crop at $\sigma = 50$: clean, noisy, the starting DnCNN and the fine-tuned models at $\lambda = 0$ (control), $\lambda^* = 1$ and $\lambda = 3$ (arranged as a grid from results/08/fig_crops.png).

# 8 Conclusion and Remarks

## 8.1 Conclusions

When I started this project, my main motivation was a simple question: does a restorer model really help a detector at its job, and is it PSNR that picks the restorer that helps the detector out the most? After building the degrade-restore-detect pipeline from scratch and running 64 restoration runs on 300 COCO images, the main finding is that restoration helps a great deal when matched to the right damage, but a high PSNR does not automatically translate to something better for the detector.

Looking at the three types of damage in stage 1 showed why no single restorer model works best everywhere. On Gaussian noise, DnCNN and BM3D both recovered about three-quarters of the lost mAP (with DnCNN recovering 77% at $\sigma = 25$ and 72% at $\sigma = 50$), ending up in a statistical tie on detection even though DnCNN scored higher on PSNR. That told me the detection gain comes mainly from removing the noise well without losing edges and textures, rather than from using a learned network, although in practice DnCNN is blind and runs the 300 images in 45 seconds on a GPU compared to 1.8 hours per noise level on four CPU cores for BM3D. Checking the internals of my trained DnCNN helped explain how it works: 43 of its 64 first-layer filters act on differences between colour channels, and its effective receptive field widens as the noise grows (from $\pm 7$ pixels at $\sigma = 15$ and 25 to $\pm 10$ at $\sigma = 50$), allowing it to keep finer details than the best classical filter. On JPEG compression, however, DnCNN did not help at all ($-5\%$ recovery at $q = 30$ and $-2\%$ at $q = 10$) and produced streaks along strong edges because it never saw block artifacts during training, whereas the bilateral filter recovered 48% and 41% of

the lost mAP at strong compression (q = 30 and 10) by smoothing out the $8 \times 8$ block edges that the detector would otherwise treat as real edges.

In low light, PSNR misled us. By testing darkened images without added noise in stage 2, I learned that the shot and read noise, rather than the darkness itself, is what actually hurts the detector: noise-free dark images kept 0.301 mAP (against 0.121 with noise), and a simple gamma curve brought them right back to 0.383. On full low-light damage, CLAHE raised PSNR (from 8.13 to 10.37 dB at $\ell = 0.8$) but made detection worse than doing nothing at every level ($-18\%$ at $\ell = 0.6$ and $-10\%$ at $\ell = 0.8$) because it brightened the noise along with the image. Combining gamma brightening with DnCNN worked best, and the order of the two steps changed the outcome: gamma $\rightarrow$ DnCNN reached 0.242 mAP at $\ell = 0.8$, while DnCNN $\rightarrow$ gamma got 0.196 despite having a higher PSNR (16.71 vs 16.17 dB). Brightening first likely lifts the noise into the range DnCNN was trained to undo, whereas denoising first leaves faint noise behind that the gamma curve then amplifies.

Now, with all that in mind, how well did PSNR track detection overall? It was fine on average for Gaussian noise (only 1 of 24 pairs reversed) and JPEG (2 of 12), but it reversed 63 of 144 low-light pairs and said almost nothing about individual images, where the correlation between per-image PSNR gain and AP gain had a median of just 0.03 across 52 runs. Even at $\sigma = 50$, where DnCNN improved AP on 91% of the images, it lowered AP on 5% of them, and nothing in the pixels showed which ones. That is why I looked inside the detector in stage 2.

By hooking every layer of YOLOv8n and measuring how far the damaged and restored feature maps were from the clean ones, I saw that the detector does not filter damage out as it goes deeper: Gaussian noise damage at the neck is about as high as at the first layer, and JPEG damage grows with depth to 3.33 times its layer-0 distance at q = 10. More importantly, DnCNN removed 82–93% of the pixel error on Gaussian noise, but only 58–62% of the feature damage at the neck, leaving roughly 40% of the neck damage behind. Measuring feature distance at the neck accounted for the BM3D and DnCNN tie (their neck distances were within 12–15% of each other), picked the best restorer in all 7 conditions while PSNR picked it in 6 (failing at $\ell = 0.8$), and tracked per-image AP loss much better than PSNR across 35 of 36 runs (averaging 0.299 vs $-0.08$).

In stage 3, I tested whether fine-tuning DnCNN to repair those layer-21 features would help the detector out more. At heavy noise ($\sigma = 50$), the added feature loss gave a small gain over the pixel-only control: YOLOv8n mAP rose from 0.302 to 0.312 at the validation-chosen $\lambda^* = 1$, and reached 0.317 at $\lambda = 0.3$, winning back about 16% of the remaining lost mAP at the cost of 0.1 to 0.4 dB of PSNR. Fine-tuning on COCO also taught the denoiser to leave clean images almost untouched (raising clean PSNR from 37.5 dB to 39–46 dB), and the high-frequency share of the restoration error stayed unchanged (0.550 vs 0.552), suggesting that the denoiser was not fooling YOLOv8n with adversarial patterns. The gain carried over to YOLOv8s, which was never used in training (+0.011 to +0.019 image-averaged AP), while for

Faster R-CNN the mAP gain was similar in size (0.351 to 0.367 at $\lambda^* = 1$) but wasn't statistically clear at the chosen $\lambda$ because its per-image interval narrowly included zero. Across the last two stages, 8 of 13 hypotheses held in stage 2 and 2 of 4 held in stage 3. Because the thresholds were written down before each experiment was run, I reported the near-misses (such as 0.299 against a 0.30 threshold for neck correlation, and +0.0096 against +0.010 mAP at $\lambda^* = 1$) as failures.

If I had to give one piece of advice to someone choosing a restorer for a detector, I would tell them not to pick the model that scores best on PSNR, but to judge it directly by the detector's own mAP or by how well it repairs the features inside the detector's neck.

### 8.2 Limitations and Future Work

Several limitations of this study point directly to what I can do next. First, only one main detector (YOLOv8n) and a 300-image subset of COCO were used, which leaves small differences like the tie between DnCNN and BM3D unresolved, and the 500-image validation set in stage 3 was too small to cleanly separate the $\lambda$ values. Second, all the damage was synthetic: real sensor noise is signal-dependent and spatially correlated, and real low light is more complex than a tone curve plus noise. Because DnCNN was trained only on Gaussian noise, its results on JPEG and low light measure out-of-distribution generalisation rather than what a restorer model trained on that specific damage could achieve. Also, the tuned classical filters and BM3D acted as oracles because they were given the true noise level or tuned on validation images beforehand, which favours them over my blind DnCNN restorer model and makes its margins over them conservative (while BM3D's 1.46 dB gap to its published value at $\sigma = 50$ is likely due to clipping the pixels to [0, 1], which was not tested). Finally, because official COCO mAP pools detections across the whole dataset and cannot be cheaply recomputed thousands of times, the paired bootstrap intervals had to use image-averaged AP, i.e. the mean of each image's own AP, as a stand-in for COCO mAP.

In future work, the next experiments I would prioritise are training the restorer model on JPEG and low-light damage and extending the detector-feature loss to them, using a larger validation set to choose $\lambda$, and testing whether pixel clipping explains BM3D's gap at $\sigma = 50$. Beyond that, I want to test the pipeline on real damaged photographs across more detectors, and compare image restoration directly against fine-tuning the detector itself on damaged images. Finally, building the full pipeline from scratch, including the seeded damage models with automated checks, the NumPy JPEG codec and convolution layer, the DnCNN, and the pre-registered hypotheses, gave me a solid foundation to take these next experiments forward.

# References


Arbeláez, P., Maire, M., Fowlkes, C., & Malik, J. (2011). Contour detection and hierarchical image segmentation. IEEE Transactions on Pattern Analysis and Machine Intelligence, 33(5), 898–916.

Dabov, K., Foi, A., Katkovnik, V., & Egiazarian, K. (2007). Image denoising by sparse 3-D transform-domain collaborative filtering. IEEE Transactions on Image Processing, 16(8), 2080–2095.

Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8 [Software]. https://github.com/ultralytics/ultralytics

Kornblith, S., Norouzi, M., Lee, H., & Hinton, G. (2019). Similarity of neural network representations revisited. In Proc. 36th International Conference on Machine Learning (ICML), PMLR 97, 3519–3529.

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In Proc. European Conference on Computer Vision (ECCV), 740–755.

Liu, D., Wen, B., Liu, X., Wang, Z., & Huang, T. S. (2018). When image denoising meets high-level vision tasks: A deep learning approach. In Proc. 27th International Joint Conference on Artificial Intelligence (IJCAI), 842–848.

Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. In Advances in Neural Information Processing Systems (NeurIPS) 28, 91–99.

Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). Image quality assessment: From error visibility to structural similarity. IEEE Transactions on Image Processing, 13(4), 600–612.

Zhang, K., Zuo, W., Chen, Y., Meng, D., & Zhang, L. (2017). Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising. IEEE Transactions on Image Processing, 26(7), 3142–3155.

Zhang, K., Zuo, W., & Zhang, L. (2018). FFDNet: Toward a fast and flexible solution for CNN-based image denoising. IEEE Transactions on Image Processing, 27(9), 4608–4622.

Zuiderveld, K. (1994). Contrast limited adaptive histogram equalization. In P. S. Heckbert (Ed.), Graphics Gems IV (pp. 474–485). Academic Press.


# Use of AI Assistance

I was mainly interested in the topic of denoising, and it is what led me down this rabbit hole. My curiosity in this field led me to spend most of my time running experiments and getting results, and with limited time left for writing, I used AI to fill those gaps in the report. An AI assistant (Claude, Anthropic) was used throughout this project. In the experiments, it helped plan the studies, scaffold and debug the notebooks, and write analysis scripts. I ran notebooks 01, 04, 05, 05b, 06, 07 and 08 on Kaggle, and notebooks 02 and 03 were run on a CPU outside Kaggle. In this report, I wrote the Introduction, Sections 4–6 and Section 8 from outlines and fact sheets provided by the assistant, which also corrected grammar and formatting, made small wording fixes in the Introduction and Section 4.1, condensed Section 4.4 and supplied some short factual sentences at my request. The assistant drafted the Abstract, Sections 2, 3 and 7 and the reference list, which I then edited, and it revised their wording after a writing review. It also made the cover page and table of contents, drew the block diagrams (Figures 1, 2, 4 and 5), prepared Figures 6, 8 and 13 from the saved results, numbered the tables and figures, and assembled the combined code repository and its documentation. The other figures are outputs of the project notebooks. Every number in the report was checked against the saved result files, and the references against the original papers.